

Group 2: Ring Oscillator/SRAM PUF

Yesnas Gowda, Rahul Narasimhegowda, Avinash Gupta, Bao Vo, Ali Hammoud, Alex Knowlton

Abstract

This project involves the design and integration of a unique PUF (Physically Unclonable Function) generator into a 16-bit microprocessor. PUFs are increasingly important for uniquely identifying devices for Internet-of-Things (IoT) applications such that devices can be securely identified and linked. This project involves designing and comparing two different PUF topologies. Two designs are compared: a mixed-signal ring oscillator PUF and an SRAM PUF. Both circuits are compared at the schematic level. The PUF is tied into the datapath of the microprocessor. A custom PUF instruction allows the processor to retrieve the PUF value when necessary, and four challenge bits help with temperature variation, uniqueness, and robustness. The fascinating part of the PUF is that the circuit relies on random circuit variation for correct operation rather than being badly impacted by it.

Introduction

PUFs rely on the random variation of device processing to generate unique keys for each fabricated chip. This means that each key must be unique to the chip but be entirely consistent for all operating conditions of that chip. It is also possible for PUFs to be implemented with challenges such that they can have multiple keys for security or higher yield. Two PUFs are compared. The first, a ring oscillator PUF, uses random mismatches in devices to compare frequencies of two ring oscillators and output a 1 or a 0 based on frequency comparison. The other design, an SRAM PUF, uses random mismatch to generate a stored 1 or 0. Challenge bits are used post-fabrication to select the most reliable PUF generator(s) in the array so that the

impact of temperature variation can be dramatically reduced. Accounting for temperature variation is the most challenging part and is addressed in Design.

Design and Implementation

Methodology:

Ring oscillators are very susceptible to variations in process and mismatch which makes them great for generating physically unclonable functions. However, this also means that they are also very sensitive to temperature variations. An interesting approach that our work takes to avoid ring oscillator temperature dependence is by using a phase frequency detector (PFD) to compare two ring oscillator frequencies. This effectively cancels the any temperature variation given the assumption that the relative frequencies is preserved. The PFD drives a simple charge pump. Depending on which ring oscillator is faster, the charge pump charges or discharges a capacitor. The value of the cap is sampled using a finite state machine to generate one PUF bit. Figure 1, describes all the different circuits used by the ring oscillator PUF.

Ring oscillator:

We are using a five stage, four-stacked inverter for the ring oscillator core. The devices are made long to lower the frequency. The reason for this is covered in the PFD section. The first gate in the ring oscillator is a NAND gate that is used to gate the ring oscillator to save power. The upper PMOS of the ring oscillator is controlled by a control voltage that is used to compensate for the frequency drift with temperature. This compensation further helps stabilize the PUF across temperature. [1].

Reference Generation:

We use a PTAT to generate a reference voltage to help compensate for the decrease in ring oscillator current with temperature. To give us more fine control of this reference we add a

weak CTAT reference is parallel to this.

Phase Frequency Detector:

The phase frequency detector is especially challenging to design because of the high frequency of operation of the ring oscillators. We used several approaches to help in this regard.

1. Use of LVT devices for the PFD. (This helps reduce delay)
2. Lowering the ring oscillator frequency.
3. Decreasing the charge pump current.
4. Increasing the Output Capacitor size.

A unique property of the architecture of the Phase Frequency Detector is that it is able to discern between the frequencies of the inputs and not just the phases like a standard XOR based detector. Figure 3 (a) shows the phase vs current plot. Figure 3(b) demonstrates the transient pulses between two different input frequencies.

Charge Pump and Capacitor:

Since we only expect UP/ DOWN pulses from the PFD, the mismatch of the chargepump or the value of the capacitor does not matter as long as we give the circuit sufficient time to settle. This simplifies the design of the chargepump and allows us to use a mos-cap for the capacitance.

State Machine:

Generating the necessary 4x 16 bits for a single PUF value requires 12 unique oscillators. Each comparison requires two ring oscillators. Since each additional ring oscillator can be compared with previous ring oscillator, the number of PUF bits is given by $N_{bits} = \sum_{i=1}^n (i - 1)$, where n is the number of ring oscillators. The PFD, Charge Pump and Capacitor is reused for all

different PUF combinations, allowing a much smaller area. The state machine selects two frequencies to send to the PFD and uses a counter (count 64) to wait for the capacitor charge to stabilize. It subsequently reads the values and continues until all 16 bits are computed for the given challenge. 4 challenges are implemented with different combinations of inputs.

Alternative Design (SRAM PUF)

We also explored the design of an SRAM based PUF and compared its performance. We additionally implemented In-memory computing on a 16x16 SRAM.

- **PUF Generation:** Currently, the PUF is generated using a traditional approach — powering up the SRAM cell and reading the resultant value, which due to inherent physical variabilities, is unique to each cell. Cells which are not voltage and temperature invariant will be masked and not used for key generation [2].
- **Encryption and In-Memory Computing:** Once the key (PUF) is generated, it is used for encrypting weights used in in-memory computing. The basic operation involves XORing this key with the weights before computation, embedding encryption into the initial phase of data processing. This eliminates the need for a separate encryption process after data generation. For in-memory computing, we need to be able to read the cell values. For this purpose, the sense amplifier is implemented. The Sense Amplifier design involves DCVS feedback to enable fast reading [3].

Results

The SRAM PUF was more functional since binary compute-in-memory (CIM) was added in an additional row of cells on top of the row of cells for generating PUF values. However, serious issues with temperature sensitivity prevented the PUF from being useable in this case. The ring oscillator PUF was temperature-compensated, which has additional design time and higher static power. The control logic for the RO PUF was also substantially more complex, and it has a lag time before being readable.

	Yield	Size	Power	Lag Time	% of 1's
Ring Oscillator PUF	95% ¹	6072um ²	335uW ²	10us	54%
SRAM PUF	~10%	8110um ²	1.1mW	Instant	~50%

This comparison begs the question of which PUF is more suitable in different circumstances. The ring oscillator PUF is clearly smaller than the SRAM PUF and consumes much less power, but is also has a lag time for generation. The SRAM PUF, on the other hand, is nearly instant, and has the additional benefit of being able to do compute-in-memory (CIM), which could potentially be used for encryption using the PUF or by built-in-self-test (BIST) to self-mask and improve the yield. For this project's purposes however, the SRAM yield was too low to be feasible. If the SRAM were much larger so that it had a higher chance of a successful PUF value being generated that was temperature-insensitive, then it would be more suitable, especially for very new designs that have large amounts of silicon that is not often in use. Another reason to use the SRAM PUF is that the spread of 1's and 0's is more even, leading to a higher Hamming distance, which is a common PUF metric. However, in a small, relatively low-power design like this, the RO PUF is more useful.

¹ Yield due to lack of settling time due to near-identical ring oscillators

² Power consumed while PUF generates. After generation, the power drops to 118 uW.

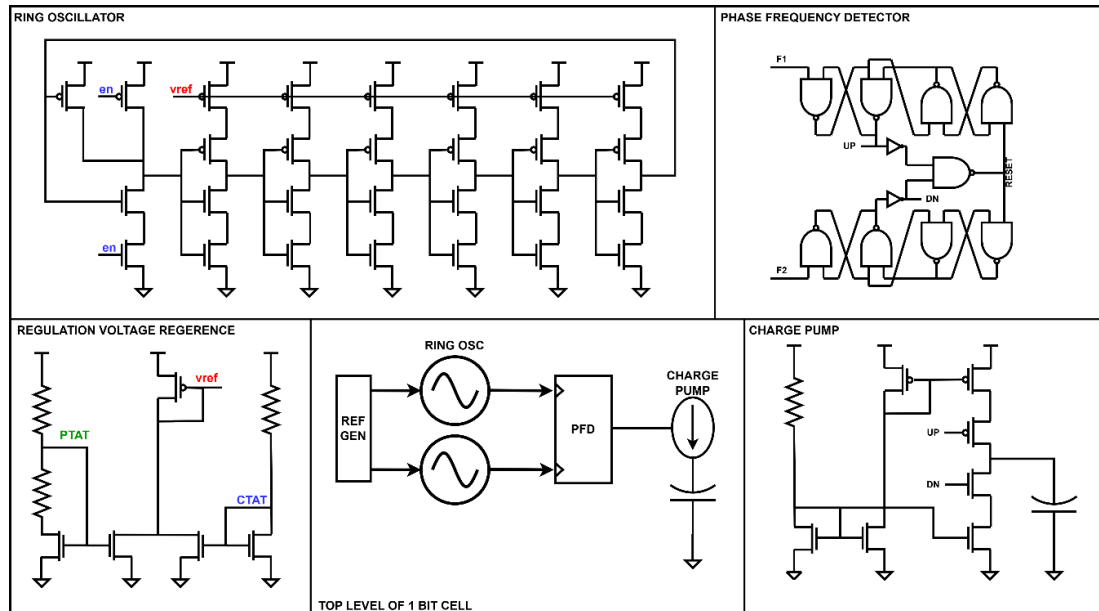


Figure 1: Ring Oscillator based PUF Circuit Design and Implementation

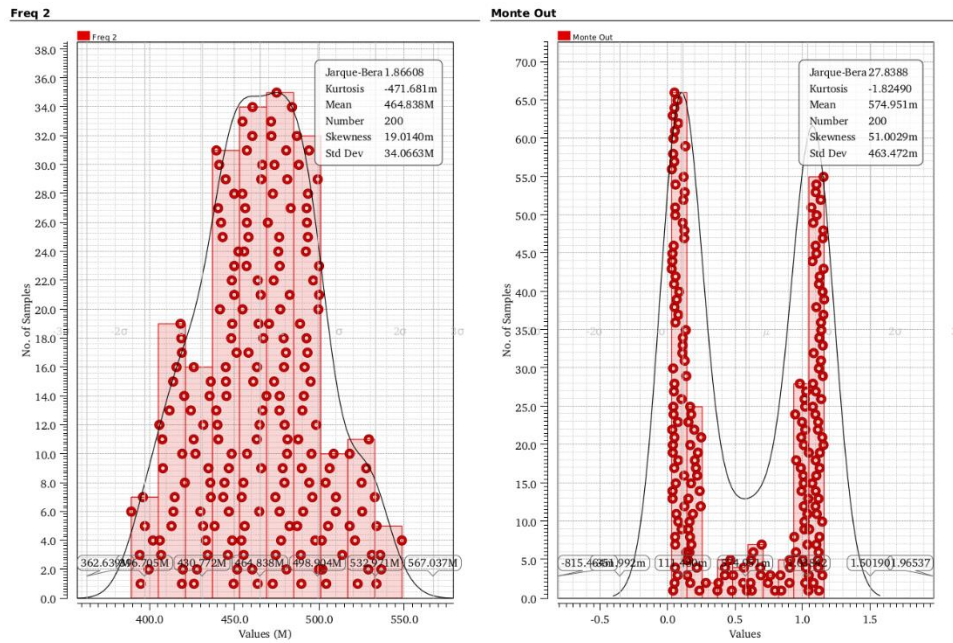


Figure 2: Ring Oscillator based PUF Monte Carlo statistics of the frequency deviation of ring oscillator (a) and the output bit value (b).

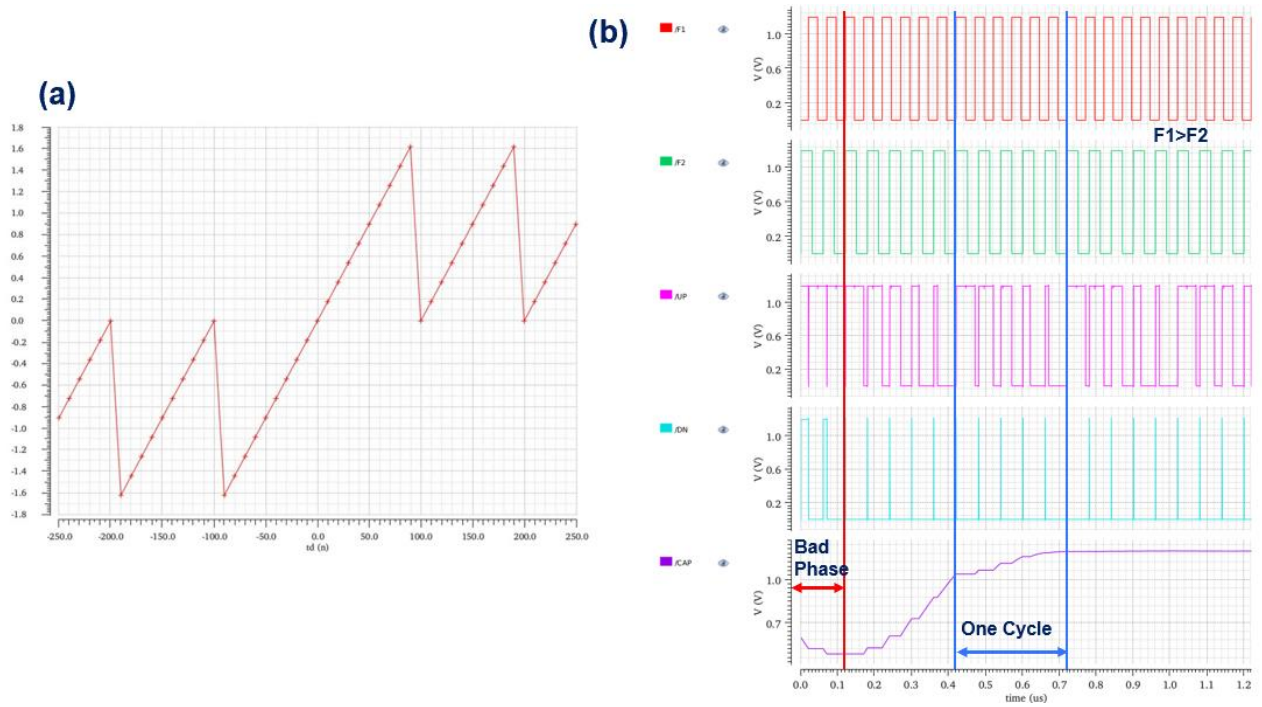


Figure 3: (a) PFD Phase vs Pulse width. (b) PFD Transient Simulation showing that across multiple cycles the output is invariant even with an initial negative spike due to phase mismatch.

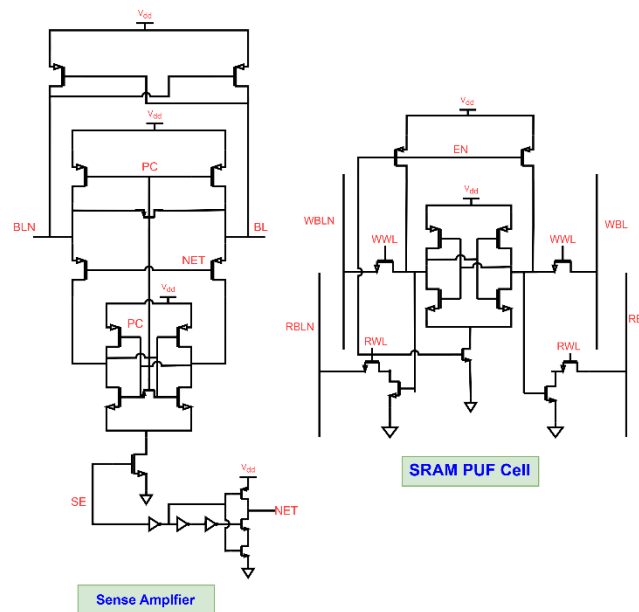


Figure 4: Sense Amplifier and SRAM based PUF Circuit Design and Implementation

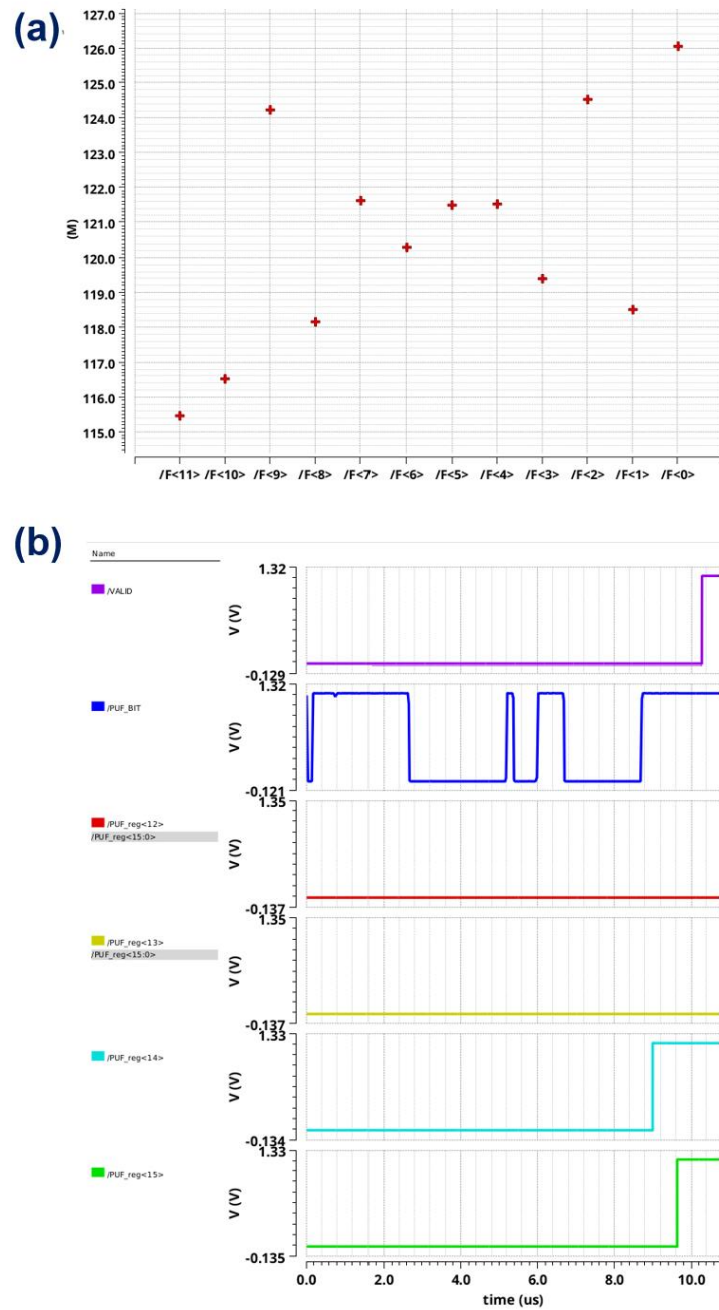


Figure 5: (a) Scatter plot of ring oscillator frequencies, (b) PUF simulation with the valid bit going high after all 16 PUF registers are populated. 4 PUF register bits are shown. Note that the PUF register bits are the time-sampled bits of the PUF_BIT value, which is multiplexed before being latched.

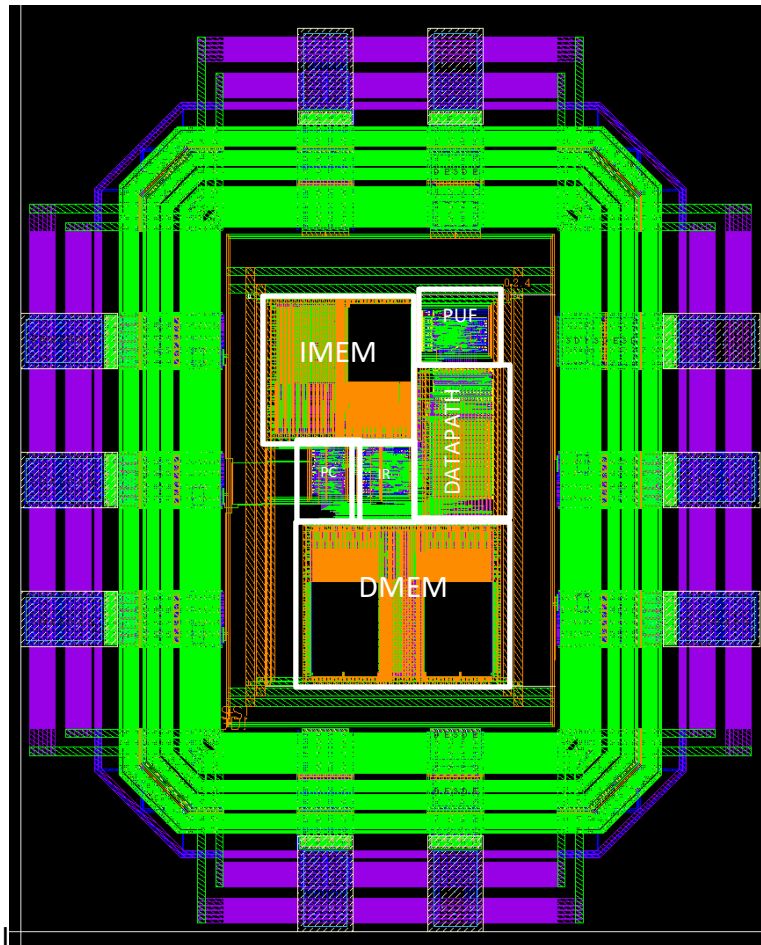


Figure 6: Circuit layout with IO pads, and custom block in the center. The total area is 0.99 mm^2 , and the area of the core is 0.135 mm^2 . For the library in Cadence, see [/afs/umich.edu/class/eecs427/f23/group2/CAD8_copy_alex_v2](https://afs.umich.edu/class/eecs427/f23/group2/CAD8_copy_alex_v2), where there is a README and a submit folder with NCVerilog waveforms of the entire design.

References

- [1] Vivekraj, V. and Nazhandali, L., 2011, January. Feedback based supply voltage control for temperature variation tolerant PUFs. In 2011 24th International Conference on VLSI Design (pp. 214-219). IEEE.
- [2] Y. Shifman, A. Miller, Y. Weizmann, and J. Shor, "A 2 Bit/Cell Tilting SRAM-Based PUF With a BER of $3.1\text{E}-10$ and an Energy of 21 fJ/Bit in 65nm," in IEEE Open Journal of Circuits and Systems, vol. 1, pp. 205-217, 2020, doi: 10.1109/OJCAS.2020.3034266.
- [3] Y. -P. Tao and W. -P. Hu, "Design of Sense Amplifier in the High-Speed SRAM," 2015 International Conference on Cyber-Enabled Distributed Computing and Knowledge Discovery, Xi'an, China, 2015, pp. 384-387, doi: 10.1109/CyberC.2015.32.